

《面向对象程序设计（JAVA）》综合实验报告

题目： 2048 游戏

学生姓名 苏富杰 电话 13543613961

学 号 202544031127

学 院 计算机科学与工程学院

专 业 信息管理与信息系统

班 级 4251

成 绩 _____

日 期 6.22-6.26

小组其他成员：

班 级	学 号	姓 名	联系电话

广东海洋大学学生综合实验报告

一、需求分析

1.1 程序设计任务

本综合实验的任务是设计并实现一款基于 Java 的 2048 数字益智游戏。游戏在 4×4 棋盘上进行，玩家通过键盘方向键控制数字方块滑动，相同数值方块碰撞后合并为两倍数值的方块，合并所得数值即为玩家获得的分数。游戏目标为合成 2048 方块；若棋盘所有格子被填满且不存在可合并的相邻方块，则游戏结束。选取 2048 作为选题的原因在于：规则简明但算法有深度，涵盖数组操作、状态管理、事件驱动编程等核心技术点，能够充分展示面向对象程序设计中的封装、继承、多态、组合以及 MVC 设计模式的综合运用。

1.2 开发平台与工具

开发语言选用 Java (JDK 21)。图形用户界面采用 JavaFX 框架，使用 Canvas 组件进行棋盘渲染，BorderPane、HBox、VBox 等布局管理器构建界面结构。数据库选用 MySQL 8.0，通过 JDBC 进行连接与操作。MySQL 是目前应用最广泛的关系型数据库之一，使用它能够展示 JDBC 编程中的连接管理、PreparedStatement 防 SQL 注入、结果集处理等技能。开发工具使用 IntelliJ IDEA 集成开发环境。

1.3 输入形式与输入范围

程序的输入包括键盘按键事件和鼠标点击事件。键盘操作支持两套键位方案：方向键（↑↓←→）和 WASD 键。鼠标操作为点击界面上的“新游戏”按钮和“撤销”按钮。程序仅响应上述合法输入，其他按键将被忽略。

1.4 输出形式

程序的输出包括五个方面：（1）图形界面输出——Canvas 渲染 4×4 棋盘，圆角矩形方块，不同数值对应不同背景色和字体颜色；（2）分数输出——SCORE 和 BEST 计分框实时显示当前得分和历史最高分；（3）状态提示——底部状态栏显示游戏进行状态和操作反馈；（4）对话框输出——胜负判定时弹出 Alert 对话框；（5）数据库输出——游戏结束或窗口关闭时自动将数据写入 MySQL 数据库的 high_score 表和 game_record 表。

1.5 程序功能

本游戏程序共实现以下八项功能：

功能一（四方向移动与合并）：支持上下左右四个方向的方块滑动，相同数值方块碰撞后合并为两倍数值的方块。

功能二（实时分数追踪）：每次合并累加分数，SCORE 实时显示当前得分，BEST 显示历史最高分。

功能三（胜负判定与继续挑战）：棋盘出现 ≥2048 方块时弹出胜利对话框询问是否继续；棋盘满且无合法移动时判定失败。

功能四（撤销操作）：可撤销上一步移动，棋盘状态和分数同步恢复到操作前。

功能五（双键位支持）：同时支持方向键和 WASD 键两种操作方案。

功能六（颜色编码方块）：不同数值使用不同背景颜色，数值越大颜色越深，小数字深色字体、大数字白色字体。

功能七（新游戏）：可随时重置棋盘和当前分数，最高分保留。

功能八（MySQL 数据库持久化）：程序启动时从数据库加载历史最高分；游戏结束时自动将本局得分、最大方块值、操作步数、胜负结果存入 `game_record` 表；当得分超过历史最高分时更新 `high_score` 表。数据永久保存，可通过 Navicat 等工具查看。

1.6 测试数据

设计以下测试用例验证程序功能：

测试 1（基本移动）：初始棋盘有两个方块，按方向键。预期：方块滑动，新方块生成。

测试 2（合并）：相邻相等方块滑动。预期：合并为两倍数值，SCORE 增加。

测试 3（防链式合并）：`[2, 2, 4, 0]` 左移。预期：结果为 `[4, 4, 0, 0]` 而非 `[8, 0, 0, 0]`。

测试 4（无效移动）：方块贴边时继续滑动。预期：棋盘和分数不变。

测试 5（WASD 键）：分别按 W/A/S/D。预期：对应上/左/下/右移动。

测试 6（撤销）：移动后点撤销。预期：棋盘和分数恢复。

测试 7（胜负判定）：达成 2048 或棋盘满。预期：弹出对应对话框。

测试 8（数据库）：完成游戏后重启程序。预期：最高分保留，Navicat 可见 `game_record` 新记录。

二、概要设计

2.1 系统架构设计

本系统采用 MVC 设计模式进行架构规划。模型层 (Model) 由 `Tile`、`Board`、`GameState`、`Direction` 和 `DatabaseManager` 五个类组成，负责游戏数据和业务逻辑的管理。视图层 (View) 由 `GameView` 类实现，基于 JavaFX 框架构建图形用户界面。控制器层 (Controller) 由 `GameController` 类实现，作为 Model 和 View 之间的中介者，负责接收用户输入、调度业务逻辑、刷新界面显示和数据持久化。MVC 模式使各层职责清晰、耦合度低，便于后续维护和功能扩展。

2.2 类结构设计

系统共包含八个 Java 源文件（含一个枚举类型），全部位于 `game2048` 包中：

(1) `Main` 类：程序入口，继承 JavaFX 的 `Application` 类，在 `start` 方法中创建 `Stage` 并实例化 `GameController`。

(2) `Direction` 枚举：定义 `LEFT`、`RIGHT`、`UP`、`DOWN` 四个方向常量，提供类型安全的参数表示。

(3) `Tile` 类：方块数据模型，封装 `value`（数值，0=空位）和 `merged`（本轮合并标记）属性。`merged` 标志是防链式合并机制的核心。

(4) `Board` 类：棋盘核心逻辑类，管理 `4×4` 的 `Tile` 二维数组，实现四方向移动合并算法、随机生成算法（90% 概率 2、10% 概率 4）、胜负判定算法以及撤销快照机制（深拷贝）。

(5) `GameState` 类：游戏状态管理类，维护 `score`（当前分数）、`bestScore`（历史

最高分)、gameOver、won、keepPlaying 等状态信息。

(6) GameView 类: JavaFX 视图类, 使用 Canvas 绘制棋盘、BorderPane 管理布局、HBox 和 VBox 组织组件、Alert 实现对话框。封装了 2048 经典配色方案和自适应字号渲染。

(7) GameController 类: 控制器类, 处理键盘和按钮事件, 协调 Model 和 View 之间的数据流动。新增了步数统计 (steps) 和最大方块追踪 (maxTile) 功能, 在游戏结束和窗口关闭时通过 DatabaseManager 将数据持久化到 MySQL。

(8) DatabaseManager 类: MySQL 数据库管理类, 封装 JDBC 连接和 CRUD 操作。负责最高分存取 (getHighScore/saveHighScore)、游戏记录保存 (saveGameRecord) 和历史查询 (getRecentRecords)。使用 PreparedStatement 防止 SQL 注入, try-with-resources 管理资源释放。

2.3 程序流程设计

启动流程: Main.main() 启动 JavaFX → GameController 构造时依次初始化 DatabaseManager (连接 MySQL 并加载历史最高分)、Board (生成两个初始方块)、GameState (传入历史最高分)、GameView (构建界面) → 注册键盘和按钮事件处理器 → 配置窗口关闭事件 → 显示窗口并获取键盘焦点。

移动流程: 按键触发 handleKeyPress → processMove → 保存快照和分数 → Board.move() 执行移动合并 → 若棋盘变化: 累加分数、更新最大方块值、步数+1、生成新方块、刷新界面、检查胜负状态 (获胜或失败时自动调用 DatabaseManager 保存记录到 MySQL)。

关闭流程: 窗口关闭事件触发 → 若本局有操作则保存最高分到 high_score 表 → 保存本局完整记录到 game_record 表 → 关闭数据库连接。

三、详细设计

3.1 Tile 类

Tile 封装单个方块的 value(数值, 0 表示空位) 和 merged(本轮是否已合并)。merged 标志是防链式合并的关键: [2, 2, 4, 0] 左移正确结果为 [4, 4, 0, 0] 而非 [8, 0, 0, 0], 因为前两个 2 合并后的新 4 不能在本轮再次参与合并。每次移动前 Board 调用 resetMerged() 统一重置所有方块的 merged 为 false; 合并时 doubleValue() 自动设置 merged=true; mergeLine 检查合并条件时跳过 merged=true 的方块。copy() 方法提供深拷贝能力, 为 Board 的撤销快照机制提供支持。

3.2 Board 类

Board 管理 4×4 的 Tile 数组, 实现全部游戏规则。移动合并算法以左移为例: 对每一行, 提取非空方块到 List → 调用 mergeLine 合并相邻相等方块 (跳过已合并者) → 放回棋盘左对齐 → 比对原始状态判断是否变化。四个方向的移动方法共享同一个 mergeLine, 体现了代码复用。随机生成算法: 收集所有空位坐标, 随机选一个, 90% 概率生成 2、10% 概率生成 4。胜负判定: hasWon 检查是否有方块 ≥ 2048; isGameOver 先查空位、再查水平和垂直方向的相邻可合并方块。撤销快照: saveSnapshot 深拷贝全部 16 个 Tile, undo 恢复快照并重置分数。

3.3 GameView 类

GameView 基于 JavaFX 构建界面: BorderPane 根布局; 顶部 HBox 排列 SCORE 和 BEST 两个 VBox 计分框 (每个含标题 Label 和数值 Label); 中部 Canvas (440×440px) 通过 GraphicsContext 绘制深棕色底板和 16 个圆角矩形方块; 底部 VBox 包含状态 Label 和两个 Button (“新游戏”“撤销”)。配色使用 static HashMap 映射数值到 Color (从 2 的米白色到 2048 的亮金色), 数字字号根据位数自适应 (两位数 40px、三位数 34px、四位数 26px)。对话框使用 Alert 的 CONFIRMATION 类型(胜负选择)和 INFORMATION 类型(Game Over 提示)。

3.4 GameController 类

GameController 是系统的中枢。构造时依次初始化 DatabaseManager (连接 MySQL 加载历史最高分)、Board、GameState、GameView。键盘事件通过 Scene.setOnKeyPressed 绑定 handleKeyPress 方法, 将 KeyCode 映射到 Direction 并调用 processMove。按钮事件通过 setOnAction 分别绑定 newGame 和 undo 方法。窗口关闭事件(setOnCloseRequest) 触发时保存最高分和本局记录到数据库。

processMove 执行一次完整的移动周期: 检查游戏状态→保存 prevScore 和棋盘快照→Board.move(dir)→若棋盘变化则加分、更新 maxTile、steps++、生成新方块、刷新界面、检查胜负。checkGameState 在获胜或失败时调用 DatabaseManager 的 saveHighScore 和 saveGameRecord 方法, 将数据持久化到 MySQL。newGame 重置所有状态 (步数清零、maxTile 归零), undo 恢复快照和分数。refreshView 从 Model 读取数据推送到 View, 是 Model→View 的唯一通道。

3.5 DatabaseManager 类 (MySQL 数据库管理)

DatabaseManager 封装了与 MySQL 数据库的全部交互。构造时通过 Class.forName 加载 JDBC 驱动, DriverManager.getConnection 连接到本地 MySQL 的 game2048 数据库。数据库表结构 (CREATE TABLE IF NOT EXISTS) 包含 high_score 表 (id AUTO_INCREMENT PRIMARY KEY, score, update_date) 和 game_record 表 (id, score, max_tile, steps, won, play_date)。

核心方法: getHighScore() 使用 SELECT MAX(score) 查询历史最高分, 无记录返回 0; saveHighScore(score) 仅在分数超过历史记录时执行 INSERT; saveGameRecord(score, maxTile, steps, won) 将一局完整数据 INSERT 到 game_record 表, won 用 TINYINT 表示 (1=获胜, 0=未获胜); getRecentRecords(limit) 查询最近 N 条记录, 按 id 倒序。所有 SQL 操作使用 PreparedStatement 参数化查询防止 SQL 注入, 使用 try-with-resources 确保 Statement 和 ResultSet 自动关闭。close() 方法在程序退出时断开数据库连接。

四、调试分析

4.1 问题一: 链式合并错误

初次实现 moveLeft 时未引入 merged 标记, [2, 2, 4, 0] 左移后错误输出 [8, 0, 0, 0], 因为合并循环在第一次合并后继续将新生成的 4 与后面的 4 再次合并。解决方案是在 Tile 中引入 merged 字段: move 开始前统一 resetMerged(), 合并时 doubleValue() 自动标记, mergeLine 检查时跳过已合并方块。该问题说明算法设计需精确控制操作的边界条件。

4.2 问题二：撤销分数回滚不准确

撤销时通过 Board 的 scoreGained 间接计算回滚量，但 undo 后 scoreGained 值已改变导致错误。解决方案是在 Controller 层维护 prevScore 字段，每次移动前直接保存 state.getScore()，撤销时用 setScore(prevScore) 精确恢复。该修改体现了“直接保存快照优于间接推算”的设计原则。

4.3 问题三：JavaFX 键盘焦点管理

点击按钮后方向键失效，原因是按钮获得焦点导致 Scene 键盘事件无法触发。解决方案：初始化时调用 setFocusTraversable(true) 和 requestFocus() 确保棋盘区域初始焦点；按钮回调末尾再次调用 requestFocus() 归还焦点；键盘事件同时绑定到 Scene 和根节点两个层级提高可靠性。

4.4 问题四：MySQL 数据库连接失败

程序运行时出现 ClassNotFoundException (找不到 com.mysql.cj.jdbc.Driver) 和 CommunicationsException (无法连接 MySQL 服务器)。前者是因为 mysql-connector-j.jar 未加入 classpath，在 build.bat 的 javac 和 java 命令中添加 -cp 参数解决；后者是因为 MySQL 服务未启动，在运行前启动 MySQL 服务解决。此外还需确保 game2048 数据库已创建，root 用户密码正确。

4.5 调试体会

调试过程占用了较多时间，获得以下经验：逐模块测试能缩小问题范围提高效率；状态变量管理务必审慎；不同 GUI 框架的事件模型不同，需要重新理解；数据库相关错误需要分层排查（驱动加载→网络连接→权限认证→SQL 语法→业务逻辑）。

五、用户使用说明

详细的用户使用说明请参见《2048 游戏使用说明.docx》文档。此处列出基本操作步骤：

（1）环境准备：确保已安装 JDK 21、JavaFX SDK 21、MySQL 8.0，MySQL 服务已启动，game2048 数据库已创建。

（2）启动程序：双击 build.bat 自动编译运行，或在 IDEA 中配置 JavaFX 模块路径和 MySQL 驱动后运行。

（3）游戏操作：使用方向键或 WASD 键操控方块移动，点击按钮执行新游戏或撤销操作。

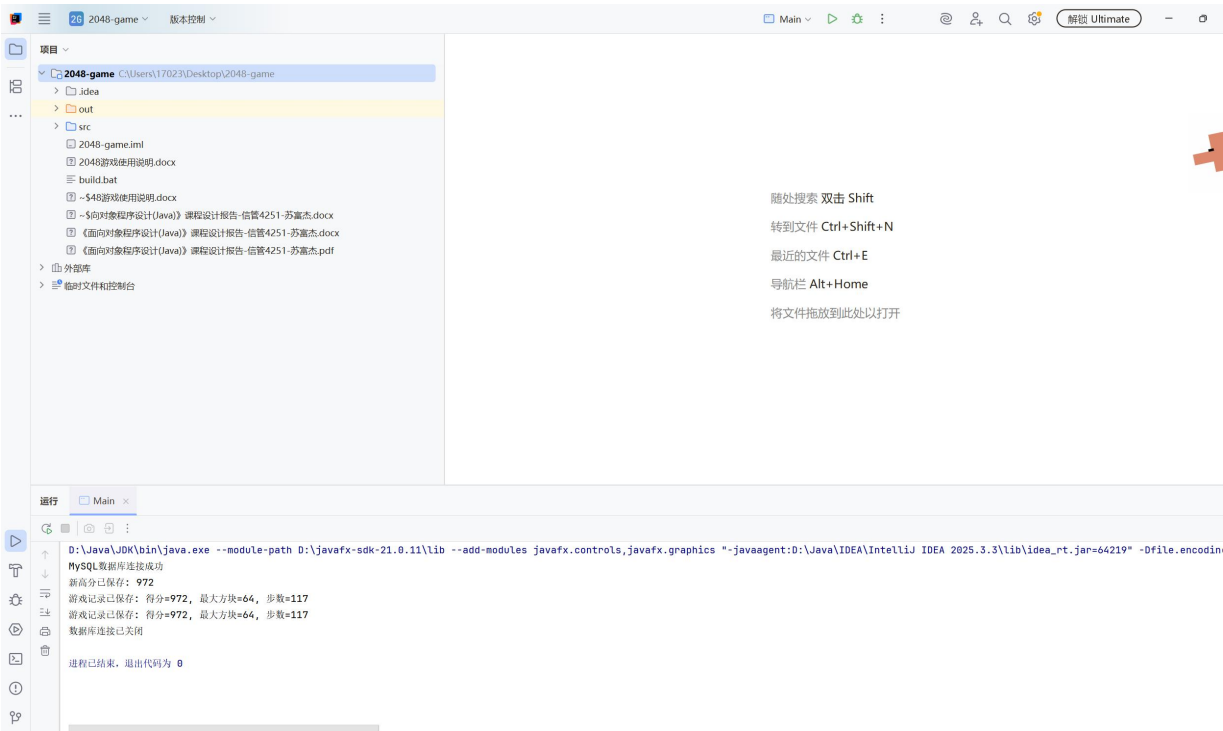
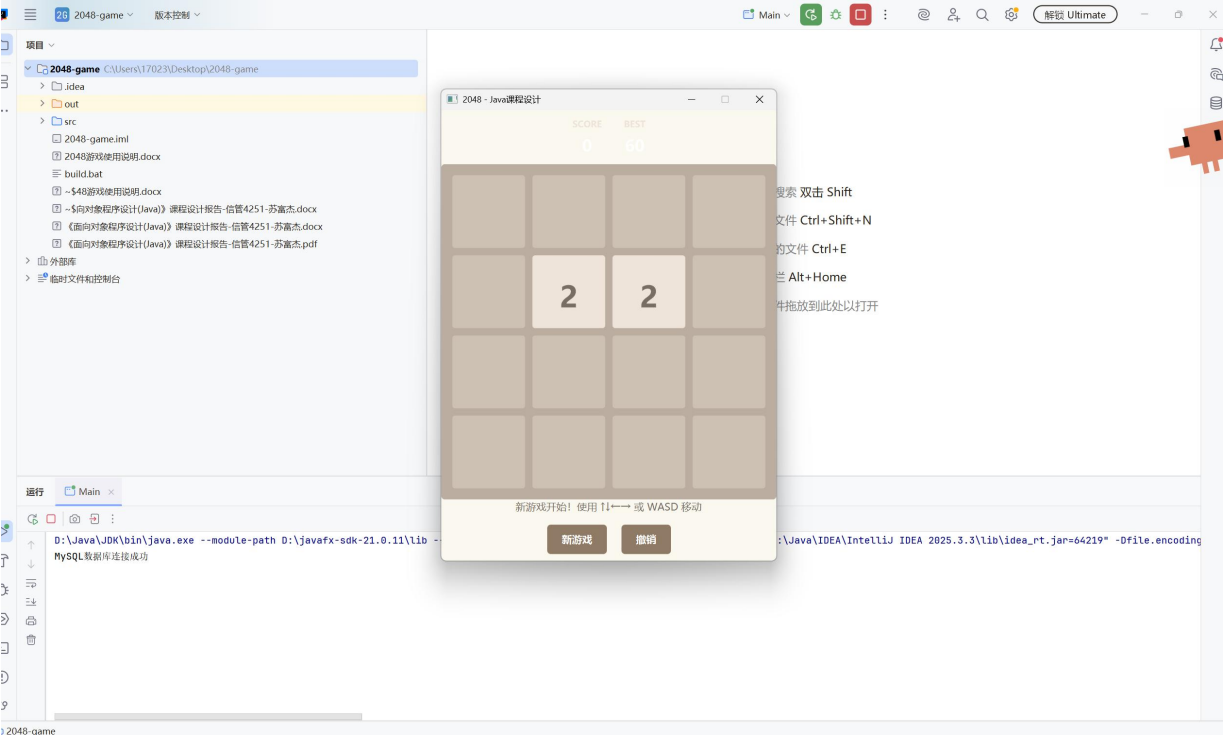
（4）数据查看：程序关闭后，使用 Navicat 连接 localhost:3306，查看 game2048 库中的 high_score 和 game_record 表。

六、测试结果

测试环境：Windows 11 Home China, JDK 21.0.10, JavaFX SDK 21.0.11, MySQL 8.0, IntelliJ IDEA 2025.3。测试日期：2026 年 6 月 24-26 日。

经 15 项功能测试，本程序的八项功能全部通过验证：四方向移动和合并逻辑正确，防链式合并机制有效，分数追踪和最高分显示准确，胜负判定符合标准规则，撤销功能

实现完整（棋盘和分数同步回滚），双键位操作等价且响应正常，数据库最高分存取和游戏记录保存无误。在连续快速操作的稳定性测试中程序运行正常，未出现崩溃或状态异常。程序达到了综合实验的功能要求和技术指标。



七、设计内容总结

7.1 系统优点

(1) MVC 架构清晰：模型、视图、控制器三层职责明确，耦合度低，便于维护和扩展。

(2) 面向对象特性运用充分：Tile 类的私有字段和公有方法体现封装；Main 继承 Application 体现继承；Controller 通过事件回调体现多态；Board 组合 Tile 数组体现组合优于继承；Direction 枚举提供类型安全常量。

(3) 核心算法实现正确：四个方向共用 mergeLine 方法实现代码复用；merged 标记机制有效解决链式合并；深拷贝快照保证撤销准确性。

(4) 数据库集成完整：使用 MySQL JDBC 实现最高分持久化和游戏记录存储，采用 PreparedStatement 防 SQL 注入，try-with-resources 管理资源释放。数据永久保存，重启程序不丢失。

7.2 不足与改进方向

(1) 撤销仅支持一步。改进方案：使用 Stack 数据结构维护操作历史，支持多步连续撤销。

(2) 缺少动画过渡效果。改进方案：引入 JavaFX Timeline 动画，为方块滑动和合并添加平滑视觉过渡。

(3) 数据库密码明文存储。改进方案：将密码改为从外部配置文件读取，提高安全性。

(4) 缺少游戏回放功能。改进方案：在数据库中记录每一步操作序列，支持对局回放和复盘分析。

7.3 实验心得

本次《面向对象程序设计（Java）》综合实验从需求分析、系统设计、编码实现到测试验证，完整地实践了一个软件项目的开发全过程。在技术层面，通过实际编码加深了对面向对象核心概念和 MVC 模式的理解；掌握了 JavaFX GUI 开发技术，包括 Canvas 自定义绘制和事件处理机制；学习了 MySQL JDBC 编程，包括连接管理、参数化查询和资源释放。在算法层面，移动合并算法的实现让自己体会到边界条件处理的重要性。在工程实践层面，数据库功能的加入使项目从单纯的桌面游戏升级为具备数据持久化能力的完整应用，为后续学习更复杂的数据库应用系统开发积累了有益经验。

附录：程序源代码

1.1 Main. java

```
package game2048;

import javafx.application.Application;
import javafx.stage.Stage;

/**
 * Main 类 — 2048 游戏应用程序的入口
```

*

* **【类的职责】**

* 作为程序的启动点，负责初始化 JavaFX 运行时并创建主舞台。

*

* **【面向对象设计要点 - 继承 (Inheritance)】**

* Main 类继承自 JavaFX 的 Application 抽象类，这是所有 JavaFX 应用的基类。

* Application 提供了完整的 JavaFX 生命周期管理：

* - init(): 初始化阶段（可选重写）

* - start(): 创建 GUI 阶段（必须重写，由 JavaFX 运行时回调）

* - stop(): 退出阶段（可选重写）

* - launch(): 静态方法，启动整个 JavaFX 应用

*

* 继承 Application 的好处：

* 不需要自己管理 JavaFX 的底层线程和事件循环，

* 只需在 start(Stage)方法中创建界面即可。

*

* **【JavaFX 启动流程】**

* Main.main(args) 调用 Application.launch(args)

* → JavaFX 运行时启动

* → 创建主 Stage 对象

* → 回调 Main.start(Stage)

* → 在 start 中创建 GameController（进而创建整个游戏）

* → 进入事件循环，等待用户操作

*

* * @author 苏富杰

* @version 1.0

*/

public class Main extends Application {

/**

* JavaFX 应用程序的启动方法

*

* 此方法由 JavaFX 运行时自动调用，传入一个已创建好的 Stage 对象。

* Stage 是 JavaFX 中的顶层窗口容器，是所有界面的载体。

*

* @param primaryStage 主舞台（JavaFX 运行时创建并传入）

*/

@Override

public void start(Stage primaryStage) {

// 创建 GameController → 初始化 Board、GameState、GameView → 显示窗口

// 之后 JavaFX 事件循环接管，所有操作由 Controller 的事件处理器响应

new GameController(primaryStage);

}

/**

```
    * main 方法是程序的真正入口
    *
    * Application.launch(args) 会：
    * 1. 启动 JavaFX 运行时环境
    * 2. 创建主 Stage 对象
    * 3. 调用 this.start(Stage)方法
    * 4. 阻塞当前线程直到窗口关闭
    *
    * @param args 命令行参数（未使用）
    */
    public static void main(String[] args) {
        launch(args); // 委托给 JavaFX Application 基类启动应用
    }
}
```

1.2 Direction.java

```
package game2048;

/**
 * Direction — 方向枚举类型
 *
 * 【类的职责】
 * 定义 2048 游戏中四个合法的移动方向常量。
 * 用于在 Board.move()、GameController.handleKeyPress()等方法中
 * 安全地传递方向参数。
 *
 * 【面向对象设计要点 - 枚举 (Enum)】
 * Java 的枚举是一种特殊的类类型，它提供类型安全的常量定义方式。
 * 相比使用 int 常量(如 0=LEFT, 1=RIGHT)或字符串(如 "left")，
 * 枚举有以下优势：
 * 1. 类型安全：编译器会检查是否传入了非法的方向值，防止写错
 * 2. 代码可读性：Direction.LEFT 比数字 0 更直观
 * 3. 防止魔法数字：避免代码中出现难以理解的硬编码数值
 *
 * @author 苏富杰
 * @version 1.0
 */
public enum Direction {
    /** 左移 — 方块向左侧滑动 */
    LEFT,

    /** 右移 — 方块向右侧滑动 */
    RIGHT,
```

```
    /** 上移 — 方块向上方滑动 */
    UP,

    /** 下移 — 方块向下方滑动 */
    DOWN
}
}
```

1.3 Tile.java

```
package game2048;

/**
 * Tile 类 — 2048 游戏中棋盘上每一个格子的数据模型
 *
 * 【类的职责】
 * 封装单个方块的数值(value)和本轮移动中的合并状态(merged),
 * 是棋盘(Board)的最小组成单元。
 *
 * 【面向对象设计要点 - 封装】
 * 所有字段均为 private, 外部代码无法直接修改方块状态,
 * 只能通过公有方法(getValue、setValue、doubleValue 等)操作。
 * 这确保了方块数据的完整性和一致性——例如合并操作只能通过
 * doubleValue()方法执行, 而不能随意修改 value 值。
 *
 * 【关键设计: merged 标志】
 * merged 字段是整个游戏防链式合并机制的核心。
 * 在标准 2048 规则中, [2, 2, 4, 0] 左移后正确结果为 [4, 4, 0, 0],
 * 而非 [8, 0, 0, 0]。原因在于前两个 2 合并产生的新 4 不应在同一次
 * 移动中再次参与合并。merged 标志确保每个方块每轮最多合并一次。
 *
 * @author 苏富杰
 * @version 1.0
 */
public class Tile {

    /**
     * 方块的当前数值
     * - 取值为 2 的幂次方 (2, 4, 8, 16, ..., 2048, ...)
     * - 取值为 0 时表示该位置为空 (无方块)
     */
    private int value;

    /**
```

```
    * 本轮移动中是否已被合并过的标记
    * - true: 本轮已经参与过一次合并, 不能再合并
    * - false: 本轮尚未合并, 可以参与合并
    * - 每轮移动开始前由 Board.resetMergedFlags() 统一重置为 false
    */
private boolean merged;

/**
 * 无参构造方法 — 创建一个空方块
 * 初始 value = 0 (表示空格), merged = false
 */
public Tile() {
    this.value = 0;
    this.merged = false;
}

/**
 * 有参构造方法 — 创建指定数值的方块
 * 用于初始化棋盘时生成 2 或 4 的方块, 以及深拷贝(undo)时复制方块
 *
 * @param value 方块的初始数值
 */
public Tile(int value) {
    this.value = value;
    this.merged = false;
}

/**
 * 获取方块当前数值
 *
 * @return 当前数值。返回 0 表示该位置为空
 */
public int getValue() {
    return value;
}

/**
 * 设置方块数值
 * 主要用于新方块生成时赋值
 *
 * @param value 新数值
 */
public void setValue(int value) {
    this.value = value;
}
```

```
/**
 * 判断该方块位置是否为空
 *
 * @return true 表示空位 (value == 0), false 表示有方块
 */
public boolean isEmpty() {
    return value == 0;
}

/**
 * 将方块数值翻倍, 并标记当前方块为"已合并"
 *
 * 该方法在两个相同数值的方块发生碰撞合并时调用。
 * 调用后将 value 乘以 2, 同时将 merged 设为 true,
 * 阻止该方块的数值在同一轮移动中再次被合并。
 *
 * 例如: 两个值为 4 的方块合并时, 其中一个调用 doubleValue()
 * 变为 8 并标记 merged=true, 另一个方块从棋盘移除。
 */
public void doubleValue() {
    this.value *= 2;
    this.merged = true; // 标记已合并, 本轮的后续检查会跳过此方块
}

/**
 * 重置合并标记为 false
 *
 * 在每次新的移动操作开始前, 由 Board.resetMergedFlags() 统一调用。
 * 确保上一轮的合并标记不会影响本轮的操作。
 */
public void resetMerged() {
    this.merged = false;
}

/**
 * 查询该方块在本轮移动中是否已被合并
 *
 * Board.mergeLine() 方法在检查两个相邻方块能否合并时,
 * 会调用此方法作为前置条件之一: 两个方块都必须未合并过。
 *
 * @return true 表示已合并 (本轮不能再次参与合并), false 表示未合并
 */
public boolean isMerged() {
    return merged;
}
```

```
}

/**
 * 创建当前方块的深拷贝（deep copy）
 *
 * 用于撤销(undo)功能的快照机制。
 * Board.saveSnapshot() 会将整个棋盘每个 Tile 都调用 copy()进行深拷贝，
 * 而不是简单的引用复制。这样撤销时恢复快照不会影响当前棋盘。
 *
 * @return 一个数值和合并状态与当前方块完全相同的新 Tile 对象
 */
public Tile copy() {
    Tile t = new Tile(this.value);
    t.merged = this.merged;
    return t;
}
}
```

1.4 Board. java

```
package game2048;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;

/**
 * Board 类 — 2048 游戏的核心逻辑层（棋盘模型）
 *
 * 【类的职责】
 * 管理 4x4 的棋盘网格，实现游戏的全部核心算法：
 * 1. 四方向移动与合并算法（move、moveLeft/Right/Up/Down、mergeLine）
 * 2. 随机方块生成算法（spawnTile）
 * 3. 胜负判定算法（hasWon、isGameOver）
 * 4. 撤销快照机制（saveSnapshot、undo）
 *
 * 【面向对象设计要点 - 组合（Composition）】
 * Board "拥有" Tile[][] 数组 — 棋盘由方块组成，这是"has-a"关系。
 * 组合优于继承：使用组合可以让 Board 灵活地管理多个 Tile 对象，
 * 而不是通过继承某个类来获得棋盘能力。
 * 每个 Tile 独立存在，通过深拷贝实现撤销功能的快照保存。
 *
 * 【面向对象设计要点 - 封装】
 * 棋盘 grid 数组为 private，外部不能直接修改棋盘状态。

```

```
* 所有对棋盘的操作都必须通过 move() 等公有方法来完成,
* 这保证了游戏规则的一致性 — 移动一定会触发合并检查,
* 合并一定会触发分数累加, 不会被绕过。
*
* @author 苏富杰
* @version 1.0
*/
public class Board {

    /** 棋盘尺寸常量: 标准 2048 为 4x4 */
    public static final int SIZE = 4;

    /**
     * 4x4 棋盘网格, game2048.Tile[SIZE][SIZE]
     * 每个格子存储一个 Tile 对象 (value=0 表示空位)
     */
    private Tile[][] grid;

    /** 随机数生成器, 用于 spawnTile 中随机选空位和随机生成 2 或 4 */
    private Random random;

    /**
     * 本轮移动是否改变了棋盘状态
     * 若为 false, 则不生成新方块、不加分
     */
    private boolean moved;

    /**
     * 本轮移动中所有合并操作累计的分数
     * 由 mergeLine() 在每次合并时累加, move() 返回后由 Controller 读取
     */
    private int scoreGained;

    /** 上一步的棋盘快照 (深拷贝), 撤销时恢复。null 表示无快照 */
    private Tile[][] snapshot;

    /** 快照时保存的分数, 撤销时用于恢复 */
    private int snapshotScore;

    /**
     * 构造方法 — 初始化 4x4 空棋盘, 并生成两个初始方块。
     * 方块生成顺序先左后右, 棋盘创建后即进入可玩游戏状态。
     */
    public Board() {
        grid = new Tile[SIZE][SIZE];
```

```
        random = new Random();

        // 初始化所有格子为空方块 (value=0)
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                grid[i][j] = new Tile();
            }
        }

        // 开局生成两个方块 (标准 2048 规则)
        spawnTile();
        spawnTile();
    }

    /**
     * 获取指定位置的方块引用 (非拷贝)
     * 返回的是 grid 中实际的 Tile 引用, 修改会直接影响棋盘
     *
     * @param row 行号, 取值范围 0~3
     * @param col 列号, 取值范围 0~3
     * @return 该位置对应的 Tile 对象
     */
    public Tile getTile(int row, int col) {
        return grid[row][col];
    }

    /**
     * 执行一次完整的移动操作
     *
     * 这是 Controller 调用的唯一移动入口。内部完成以下步骤:
     * 1. 重置所有方块的合并标记 (防链式合并)
     * 2. 将本轮得分清零
     * 3. 根据方向参数分发到对应的方向方法
     * 4. 返回棋盘是否发生了变化
     *
     * @param dir 移动方向 (LEFT/RIGHT/UP/DOWN)
     * @return true 表示棋盘发生了变化 (有方块移动或合并), false 表示没有任何变化
     */
    public boolean move(Direction dir) {
        // 新一轮移动开始, 清除上一轮的合并标记
        resetMergedFlags();

        // 重置本轮得分, 等待 mergeLine() 累加
        scoreGained = 0;
```

```
// 根据方向分发到对应的移动方法
switch (dir) {
    case LEFT: return moveLeft();
    case RIGHT: return moveRight();
    case UP: return moveUp();
    case DOWN: return moveDown();
    default: return false;
}
}

/** 获取本轮移动中获得的分数 */
public int getScoreGained() {
    return scoreGained;
}

/**
 * 在随机空位生成一个新方块
 *
 * 生成规则：
 * - 90%概率生成数值为 2 的方块
 * - 10%概率生成数值为 4 的方块
 * - 随机选择空位（所有空位等概率）
 * - 若棋盘已满则不做任何操作
 *
 * 这个概率分布参考了原版 2048 的设计：
 * 大部分时间出 2 让游戏保持稳定节奏，
 * 偶尔出 4 增加不确定性，提高游戏趣味性。
 */
public void spawnTile() {
    // 收集所有空位坐标
    List<int[]> emptyCells = getEmptyCells();

    // 棋盘已满，无法生成
    if (emptyCells.isEmpty()) {
        return;
    }

    // 随机选一个空位
    int[] cell = emptyCells.get(random.nextInt(emptyCells.size()));
    int row = cell[0];
    int col = cell[1];

    // random.nextInt(10) 返回 0~9, <9 的概率为 90%
    // 90%概率生成 2, 10%概率生成 4
    grid[row][col].setValue(random.nextInt(10) < 9 ? 2 : 4);
}
```

```
}

/**
 * 判断是否胜利 — 棋盘上是否出现≥2048 的方块
 * 遍历全部 16 个格子，有一个满足即返回 true
 *
 * @return true 表示达到 2048，游戏胜利
 */
public boolean hasWon() {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (grid[i][j].getValue() >= 2048) {
                return true;
            }
        }
    }
    return false;
}

/**
 * 判断是否游戏结束（失败）
 *
 * 判定条件（两个条件都满足才算结束）：
 * a) 棋盘上没有空位（16 格全满）
 * b) 水平方向无相邻相等方块
 * c) 垂直方向无相邻相等方块
 *
 * 只要存在空位或存在可合并的相邻方块，就能继续游戏。
 *
 * @return true 表示无法继续游戏，false 表示还能继续
 */
public boolean isGameOver() {
    // 条件 a: 有空位就一定没结束
    if (!getEmptyCells().isEmpty()) {
        return false;
    }

    // 条件 b: 检查每一行，是否有水平相邻相等方块
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE - 1; j++) {
            if (grid[i][j].getValue() == grid[i][j + 1].getValue()) {
                return false; // 有可合并的，还能玩
            }
        }
    }
}
```

```
// 条件 c: 检查每一列，是否有垂直相邻相等方块
for (int i = 0; i < SIZE - 1; i++) {
    for (int j = 0; j < SIZE; j++) {
        if (grid[i][j].getValue() == grid[i + 1][j].getValue()) {
            return false; // 有可合并的，还能玩
        }
    }
}

// 满格且无相邻相等 → 游戏结束
return true;
}

/**
 * 保存棋盘快照 — 撤销功能的核心
 *
 * 在每次移动执行前调用。
 * 对当前整个棋盘做深拷贝（每个 Tile 都 new 一份副本），
 * 同时保存本轮得分值。
 * 深拷贝是关键：不能简单保存 grid 引用，否则撤销时快照已被修改。
 */
public void saveSnapshot() {
    snapshotScore = scoreGained;

    // 深拷贝：给每个格子创建独立副本
    snapshot = new Tile[SIZE][SIZE];
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            snapshot[i][j] = grid[i][j].copy(); // Tile.copy()创建独立对象
        }
    }
}

/**
 * 撤销到上一步
 *
 * 将保存的快照恢复为当前棋盘（直接替换 grid 引用），
 * 并恢复分数。恢复后清空快照（置 null），防止重复撤销。
 * 当前仅支持撤销一步。
 */
public void undo() {
    if (snapshot != null) {
        grid = snapshot; // 直接替换整个棋盘引用
        scoreGained = snapshotScore;
    }
}
```

```
        snapshot = null;           // 清空快照，确保不可重复撤销
    }
}

/**
 * 重置棋盘 — "新游戏"功能
 * 清空全部 16 个格子（创建新的空 Tile），
 * 清空撤销快照，重新生成两个初始方块。
 */
public void reset() {
    // 清空所有格子
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            grid[i][j] = new Tile();
        }
    }

    // 新游戏不能撤销旧游戏的操作
    snapshot = null;

    // 重新生成两个初始方块
    spawnTile();
    spawnTile();
}

// ===== 以下为私有方法（内部实现细节） =====

/**
 * 重置全棋盘所有方块的合并标记
 *
 * 每次新的移动开始前调用。
 * 如果不重置，上一轮的 merged 标记会影响本轮合并判断，
 * 导致某些方块在本轮无法正常合并。
 */
private void resetMergedFlags() {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            grid[i][j].resetMerged();
        }
    }
}

/**
 * 【核心算法】左移：按行处理，每行从左到右
 *
```

```
* 算法步骤（对每一行执行）：
* 第 1 步(保存)：记录该行原始状态，用于最后判断是否有变化
* 第 2 步(提取)：按从左到右的顺序，将该行所有非空方块提取到 List 中
* 第 3 步(合并)：调用 mergeLine()对 List 中相邻相等方块进行合并
* 第 4 步(放回)：将合并后的 List 从左到右放回该行，空位补 0
* 第 5 步(比对)：比较最终结果与原始状态，判断是否变化
*
* @return true 表示该行发生了变化
*/
private boolean moveLeft() {
    boolean changed = false;

    // 逐行处理
    for (int i = 0; i < SIZE; i++) {
        // === 第 1 步：保存原始行状态 ===
        Tile[] original = new Tile[SIZE];
        for (int j = 0; j < SIZE; j++) {
            original[j] = grid[i][j].copy();
        }

        // === 第 2 步：提取非空方块（从左到右）===
        List<Tile> line = new ArrayList<>();
        for (int j = 0; j < SIZE; j++) {
            if (!grid[i][j].isEmpty()) {
                line.add(grid[i][j]);
            }
        }

        // === 第 3 步：合并相邻相等方块 ===
        mergeLine(line);

        // === 第 4 步：放回棋盘（左对齐）===
        for (int j = 0; j < SIZE; j++) {
            if (j < line.size()) {
                grid[i][j] = line.get(j);    // 放入合并后的方块
            } else {
                grid[i][j] = new Tile();    // 剩余位置填空
            }
        }

        // === 第 5 步：比对变化 ===
        for (int j = 0; j < SIZE; j++) {
            if (grid[i][j].getValue() != original[j].getValue()) {
                changed = true;
                break;    // 只要有一个不同就说明变了，无需继续比对
            }
        }
    }
}
```

```
    }
}

return changed;
}

/**
 * 【核心算法】右移：按行处理，每行从右到左
 *
 * 与 moveLeft() 的算法完全对称，区别仅在于：
 * - 提取方向：从右到左（列号 3→0）
 * - 放回方向：从右到左（右对齐）
 *
 * @return true 表示该行发生了变化
 */
private boolean moveRight() {
    boolean changed = false;

    for (int i = 0; i < SIZE; i++) {
        // 保存原始状态
        Tile[] original = new Tile[SIZE];
        for (int j = 0; j < SIZE; j++) {
            original[j] = grid[i][j].copy();
        }

        // 从右到左提取非空方块
        List<Tile> line = new ArrayList<>();
        for (int j = SIZE - 1; j >= 0; j--) {
            if (!grid[i][j].isEmpty()) {
                line.add(grid[i][j]);
            }
        }

        // 合并（同一套逻辑，四个方向共用）
        mergeLine(line);

        // 从右到左放回（右对齐）
        for (int j = 0; j < SIZE; j++) {
            int col = SIZE - 1 - j;          // 从最右边开始放
            if (j < line.size()) {
                grid[i][col] = line.get(j);
            } else {
                grid[i][col] = new Tile();
            }
        }
    }
}
```

```
    }

    // 比对变化
    for (int j = 0; j < SIZE; j++) {
        if (grid[i][j].getValue() != original[j].getValue()) {
            changed = true;
            break;
        }
    }
}

return changed;
}
```

```
/**
 * 【核心算法】上移：按列处理，每列从上到下
 *
 * 与左右移的区别是遍历顺序为"列优先":
 * 外层循环遍历列号，内层按行提取该列的方块
 *
 * @return true 表示该列发生了变化
 */
private boolean moveUp() {
    boolean changed = false;

    // 逐列处理（注意这里是列优先遍历）
    for (int j = 0; j < SIZE; j++) {
        // 保存该列的原始状态
        Tile[] original = new Tile[SIZE];
        for (int i = 0; i < SIZE; i++) {
            original[i] = grid[i][j].copy();
        }

        // 从上到下提取该列的所有非空方块
        List<Tile> line = new ArrayList<>();
        for (int i = 0; i < SIZE; i++) {
            if (!grid[i][j].isEmpty()) {
                line.add(grid[i][j]);
            }
        }

        // 合并
        mergeLine(line);

        // 从上到下放回（顶部对齐）
```

```
        for (int i = 0; i < SIZE; i++) {
            if (i < line.size()) {
                grid[i][j] = line.get(i);
            } else {
                grid[i][j] = new Tile();
            }
        }
    }

    // 比对变化
    for (int i = 0; i < SIZE; i++) {
        if (grid[i][j].getValue() != original[i].getValue()) {
            changed = true;
            break;
        }
    }
}

return changed;
}

/**
 * 【核心算法】下移：按列处理，每列从下到上
 *
 * 与 moveUp() 对称，区别在于提取和放回的方向
 *
 * @return true 表示该列发生了变化
 */
private boolean moveDown() {
    boolean changed = false;

    for (int j = 0; j < SIZE; j++) {
        // 保存该列原始状态
        Tile[] original = new Tile[SIZE];
        for (int i = 0; i < SIZE; i++) {
            original[i] = grid[i][j].copy();
        }

        // 从下到上提取该列的所有非空方块
        List<Tile> line = new ArrayList<>();
        for (int i = SIZE - 1; i >= 0; i--) {
            if (!grid[i][j].isEmpty()) {
                line.add(grid[i][j]);
            }
        }
    }
}
```

```
// 合并
mergeLine(line);

// 从下到上放回（底部对齐）
for (int i = 0; i < SIZE; i++) {
    int row = SIZE - 1 - i;        // 从最底部开始放
    if (i < line.size()) {
        grid[row][j] = line.get(i);
    } else {
        grid[row][j] = new Tile();
    }
}

// 比对变化
for (int i = 0; i < SIZE; i++) {
    if (grid[i][j].getValue() != original[i].getValue()) {
        changed = true;
        break;
    }
}

return changed;
}

/**
 * 【核心算法】合并一行/一列中相邻且相等的方块
 *
 * 四个方向的移动方法都调用此方法进行合并，体现了代码复用。
 *
 * 合并规则：
 * - 遍历 List 中每个位置 i，检查 i 和 i+1 两个方块
 * - 若两者数值相等 且 都未被标记为 merged，则执行合并
 * - 合并后：第 i 个方块数值翻倍+标记 merged，第 i+1 个方块从 List 中移除
 * - 每次合并将获得的分数累加到 scoreGained
 *
 * 【防链式合并原理】
 * merged 标志确保每个方块在单次移动中最多参与一次合并。
 * 以 [2, 2, 4, 0] 为例：
 * - 提取后 List 为 [2, 2, 4]
 * - i=0: 前两个 2 数值相等且都未合并 → 第一个 2 翻倍为 4 并标记 merged，移除第二个 2
 * - 现在 List 为 [4(merged), 4]
 * - i=1: 检查时发现第一个 4 的 merged=true → 跳过，不能合并！
 * - 最终结果为 [4, 4, 0, 0]，正确！
 *
 * 若没有 merged 标志，i=1 时两个 4 又会合并，错误地得到 [8, 0, 0, 0]。
```

```

*
* @param line 待合并的方块列表（已去除了空位）
*/
private void mergeLine(List<Tile> line) {
    for (int i = 0; i < line.size() - 1; i++) {
        Tile current = line.get(i);    // 当前位置的方块
        Tile next = line.get(i + 1);    // 下一个方块

        // 合并条件：两个方块数值相等，且都未被合并过
        if (!current.isMerged() && !next.isMerged()
            && current.getValue() == next.getValue()) {

            current.doubleValue();        // 当前方块翻倍，内部自动设置 merged=true
            scoreGained += current.getValue();    // 累加分数（已翻倍后的值）
            line.remove(i + 1);            // 移除被合并的方块
        }
    }
}

/**
* 获取棋盘上所有空位（value==0 的格子）坐标
* 返回 List<int[]>, 每个元素为 {row, col}
*
* @return 空位坐标列表
*/
private List<int[]> getEmptyCells() {
    List<int[]> empty = new ArrayList<>();
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (grid[i][j].isEmpty()) {
                empty.add(new int[]{i, j});
            }
        }
    }
    return empty;
}
}

```

1.5 GameState.java

```

package game2048;

/**
* GameState 类 — 游戏状态与分数管理

```

*

* **【类的职责】**

* 管理与棋盘逻辑无关的游戏元数据：当前分数、本局最高分、

* 游戏结束标志、获胜标志、获胜后是否继续游戏等状态。

*

* **【面向对象设计要点】**

* - 单一职责：只负责分数和状态，不涉及棋盘逻辑和数据库操作

* - 封装：所有字段私有，通过公有方法访问

* - addScore() 自动更新最高分，体现了封装的价值

*

* **【数据库集成】**

* 最高分的持久化由 DatabaseManager 负责，本类不再直接读写文件。

* 构造时接收从数据库加载的历史最高分，关闭时由 Controller

* 通过 DatabaseManager 保存。

*

* @author 苏富杰

* @version 2.0

*/

public class GameState {

/** 当前游戏分数，每次合并后累加 */

private int score;

/** 本局最高分（含历史记录） */

private int bestScore;

/** 游戏是否已结束 */

private boolean gameOver;

/** 是否已达成 2048 */

private boolean won;

/** 获胜后是否继续游戏 */

private boolean keepPlaying;

/**

* 构造方法

* @param savedBestScore 从数据库加载的历史最高分

*/

public GameState(int savedBestScore) {

 this.score = 0;

 this.bestScore = savedBestScore;

 this.gameOver = false;

 this.won = false;

 this.keepPlaying = false;

```
}

/**
 * 增加分数并自动更新最高分
 * @param points 本次合并获得的分数
 */
public void addScore(int points) {
    this.score += points;
    if (this.score > this.bestScore) {
        this.bestScore = this.score;
    }
}

public int getScore() { return score; }

/** 直接设置分数 — 仅撤销时使用 */
public void setScore(int score) { this.score = score; }

public int getBestScore() { return bestScore; }

public void setGameOver(boolean gameOver) { this.gameOver = gameOver; }
public boolean isGameOver() { return gameOver; }

public void setWon(boolean won) { this.won = won; }
public boolean isWon() { return won; }

public void setKeepPlaying(boolean keepPlaying) { this.keepPlaying = keepPlaying; }
public boolean isKeepPlaying() { return keepPlaying; }

/** 重置游戏状态, 保留 bestScore */
public void reset() {
    this.score = 0;
    this.gameOver = false;
    this.won = false;
    this.keepPlaying = false;
}
}
```

1.6 GameView. java

```
package game2048;

import javafx.geometry.Pos;
import javafx.scene.Scene;
```

```
import javafx.scene.canvas.Canvas;
import javafx.scene.canvas.GraphicsContext;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
import javafx.scene.control.ButtonType;
import javafx.scene.control.Label;
import javafx.scene.layout.*;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.text.Text;

import java.util.HashMap;
import java.util.Map;
import java.util.Optional;

/**
 * GameView 类 — 2048 游戏的视图层（基于 JavaFX 框架）
 *
 * 【类的职责】
 * 负责构建和管理游戏的全部图形用户界面。
 * 包括：窗口布局、棋盘绘制、分数显示、按钮交互、对话框弹出。
 * 视图层不处理任何游戏逻辑，只负责"怎么看"。
 *
 * 【为什么选用 JavaFX? 】
 * JavaFX 是 Java 官方推荐的现代 GUI 框架，有以下优势：
 * 1. 更现代的渲染引擎，支持 CSS 样式和硬件加速
 * 2. Canvas 组件提供了简洁高效的 2D 绘图 API
 * 3. 布局系统（HBox、VBox、BorderPane）灵活方便，易于构建复杂界面
 * 4. Alert 对话框美观且易于使用
 *
 * 【面向对象设计要点 - 封装】
 * 所有界面组件（Scene、Canvas、Button、Label）都为 private，
 * Controller 只能通过公有方法（updateBoard、updateScore 等）刷新界面，
 * 不能直接操作内部组件。如果将来想换界面框架，只需修改此类。
 *
 * 【配色方案说明】
 * 采用经典 2048 游戏的配色体系，数值从小到大方块颜色由浅变深：
 * 2(米白)→4(浅驼)→8(橙色)→16~64(红色系)→128~2048(金黄色系)
 *
 * @author 苏富杰
 * @version 1.0
 */
public class GameView {
```

```
// ===== 棋盘布局常量 =====

/** 每个方格的边长（像素） */
public static final int CELL_SIZE = 100;

/** 方格之间的间距（像素） */
public static final int GAP = 10;

/** 棋盘区域与窗口边缘的距离（像素） */
public static final int PADDING = 15;

/** 整个棋盘面板的宽/高 = 15*2 + 100*4 + 10*3 = 440px */
public static final int BOARD_WIDTH = PADDING * 2 + CELL_SIZE * 4 + GAP * 3;

// ===== JavaFX 界面组件 =====

/** JavaFX 的场景图根节点，包含所有界面元素 */
private Scene scene;

/** Canvas 组件 — 用于绘制 4x4 棋盘的自定义画布 */
private Canvas canvas;

/** GraphicsContext — Canvas 的 2D 绘图上下文，提供 fillRect、fillText 等方法 */
private GraphicsContext gc;

/** 当前得分标签 — 显示在 SCORE 计分框中 */
private Label scoreLabel;

/** 本局最高分标签 — 显示在 BEST 计分框中 */
private Label bestLabel;

/** 状态提示标签 — 底部显示操作提示和游戏状态 */
private Label statusLabel;

/** 新游戏按钮 */
private Button newGameBtn;

/** 撤销按钮 */
private Button undoBtn;

/** 当前要显示的棋盘数据（由 Controller 在 updateBoard 时设置） */
private Tile[][] grid;

// ===== 配色方案（静态常量，所有实例共享） =====
```

```
/**
 * 方块数值 → 背景颜色的映射表
 * 使用 static 初始化块填充，经典 2048 配色
 */
private static final Map<Integer, Color> TILE_COLORS = new HashMap<>();
static {
    TILE_COLORS.put(0,    Color.rgb(0xCD, 0xC1, 0xB4)); // 空格 — 浅棕色
    TILE_COLORS.put(2,    Color.rgb(0xEE, 0xE4, 0xDA)); // 2 — 米白色
    TILE_COLORS.put(4,    Color.rgb(0xED, 0xE0, 0xC8)); // 4 — 浅驼色
    TILE_COLORS.put(8,    Color.rgb(0xF2, 0xB1, 0x79)); // 8 — 橙色
    TILE_COLORS.put(16,   Color.rgb(0xF5, 0x95, 0x63)); // 16 — 深橙色
    TILE_COLORS.put(32,   Color.rgb(0xF6, 0x7C, 0x5F)); // 32 — 浅红色
    TILE_COLORS.put(64,   Color.rgb(0xF6, 0x5E, 0x3B)); // 64 — 红色
    TILE_COLORS.put(128,  Color.rgb(0xED, 0xCF, 0x72)); // 128 — 浅金色
    TILE_COLORS.put(256,  Color.rgb(0xED, 0xCC, 0x61)); // 256 — 金色
    TILE_COLORS.put(512,  Color.rgb(0xED, 0xC8, 0x50)); // 512 — 深金色
    TILE_COLORS.put(1024, Color.rgb(0xED, 0xC5, 0x3F)); // 1024 — 橙金色
    TILE_COLORS.put(2048, Color.rgb(0xED, 0xC2, 0x2E)); // 2048 — 亮金色（胜利色）
}

/** 小数字 (≤4) 的字体颜色 — 深灰色，与浅色背景对比清晰 */
private static final Color FONT_COLOR_DARK = Color.rgb(0x77, 0x6E, 0x65);

/** 大数字 (≥8) 的字体颜色 — 白色，与深色背景形成对比 */
private static final Color FONT_COLOR_LIGHT = Color.rgb(0xF9, 0xF6, 0xF2);

/** 窗口整体背景色 — 米白色 */
private static final Color BG_COLOR = Color.rgb(0xFA, 0xF8, 0xEF);

/** 棋盘底板颜色 — 深棕色 */
private static final Color BOARD_BG_COLOR = Color.rgb(0xBB, 0xAD, 0xA0);

/**
 * 构造方法 — 构建整个游戏的 JavaFX 界面
 *
 * 使用 BorderPane 作为根布局，界面分为三个主要区域：
 * - 顶部(Top): 分数面板 (HBox 水平排列两个计分框)
 * - 中部(Center): Canvas 棋盘绘制区域 (440×440px)
 * - 底部(Bottom): 状态提示文字 + 按钮组 (VBox 垂直排列)
 *
 * @param onNewGame 新游戏按钮的回调 (由 Controller 提供)
 * @param onUndo    撤销按钮的回调 (由 Controller 提供)
 */
public GameView(Runnable onNewGame, Runnable onUndo) {
```

```
// ===== 1. 顶部: 分数面板 =====
// HBox: 水平布局容器, 间距 30px, 居中对齐
HBox scorePanel = new HBox(30);
scorePanel.setAlignment(Pos.CENTER);
scorePanel.setPadding(new javafx.geometry.Insets(10));

// 创建 SCORE 计分框, 取到里面的数值标签备用
VBox scoreBox = createScoreBox("SCORE");
scoreLabel = (Label) scoreBox.getChildren().get(1); // 第 1 个子节点是数值
scorePanel.getChildren().add(scoreBox);

// 创建 BEST 计分框
VBox bestBox = createScoreBox("BEST");
bestLabel = (Label) bestBox.getChildren().get(1);
scorePanel.getChildren().add(bestBox);

// ===== 2. 中部: 棋盘 Canvas =====
// Canvas 是 JavaFX 提供的位图绘制组件
// 大小为 BOARD_WIDTH×BOARD_WIDTH (440×440)
canvas = new Canvas(BOARD_WIDTH, BOARD_WIDTH);
gc = canvas.getGraphicsContext2D(); // 获取 2D 绘图上下文

// ===== 3. 底部: 状态栏 + 按钮组 =====

// 3a. 状态提示标签 (居中显示, 跨越整个宽度)
statusLabel = new Label("使用 ↑↓↔ 或 WASD 开始游戏");
statusLabel.setFont(Font.font("Microsoft YaHei", 14));
statusLabel.setTextFill(Color.rgb(0x77, 0x6E, 0x65));
statusLabel.setAlignment(Pos.CENTER);
statusLabel.setMaxWidth(Double.MAX_VALUE);

// 3b. 按钮面板 (HBox 水平排列两个按钮)
HBox btnPanel = new HBox(20);
btnPanel.setAlignment(Pos.CENTER);
btnPanel.setPadding(new javafx.geometry.Insets(5, 0, 10, 0));

// "新游戏"按钮
newGameBtn = new Button("新游戏");
styleButton(newGameBtn);
newGameBtn.setOnAction(e -> onNewGame.run()); // 点击时调用 Controller 的 newGame

// "撤销"按钮
undoBtn = new Button("撤销");
styleButton(undoBtn);
undoBtn.setOnAction(e -> onUndo.run()); // 点击时调用 Controller 的 undo
```

```
btnPanel.getChildren().addAll(newGameBtn, undoBtn);

// 3c. 将状态标签和按钮组装成 VBox
VBox bottomPanel = new VBox(10);
bottomPanel.setAlignment(Pos.CENTER);
bottomPanel.getChildren().addAll(statusLabel, btnPanel);

// ===== 4. 组装根布局 =====
// BorderPane: 五区域布局 (Top、Bottom、Left、Right、Center)
BorderPane root = new BorderPane();
root.setBackground(new Background(new BackgroundFill(BG_COLOR, null, null)));
root.setTop(scorePanel);
root.setCenter(canvas);
root.setBottom(bottomPanel);
BorderPane.setAlignment(canvas, Pos.CENTER);

// 创建场景图
scene = new Scene(root);

// 绘制初始空棋盘
drawBoard();
}

/**
 * 创建单个计分框组件
 *
 * 布局结构 (VBox 垂直排列):
 *
 * ┌──────────┐
 * │ "SCORE" │ ← 标题 Label (小字, 浅色)
 * │ "0"      │ ← 数值 Label (大字, 白色)
 * └──────────┘
 *
 * @param title 标题文字 ("SCORE" 或 "BEST")
 * @return 组装好的 VBox 计分框
 */
private VBox createScoreBox(String title) {
    VBox box = new VBox(5); // 子节点间距 5px
    box.setAlignment(Pos.CENTER);

    // 设置深棕色背景 (与棋盘底板同色), 带圆角和内边距
    box.setBackground(new Background(new BackgroundFill(
        Color.rgb(0xBB, 0xAD, 0xA0),
        new javafx.scene.layout.CornerRadii(5),
        new javafx.geometry.Insets(8, 25, 8, 25))));
```

```
// 标题: 小号加粗字体, 浅米色
Label titleLabel = new Label(title);
titleLabel.setFont(Font.font("Microsoft YaHei", FontWeight.BOLD, 12));
titleLabel.setTextFill(Color.rgb(0xEE, 0xE4, 0xDA));

// 数值: 大号加粗字体, 白色
Label valueLabel = new Label("0");
valueLabel.setFont(Font.font("Microsoft YaHei", FontWeight.BOLD, 22));
valueLabel.setTextFill(Color.WHITE);

box.getChildren().addAll(titleLabel, valueLabel);
return box;
}

/**
 * 统一样式化按钮 — 使用 CSS 内联样式
 *
 * JavaFX 支持直接在组件上设置 CSS 样式 (类似网页 CSS)。
 * 这里设置背景色#8F7A66 (深棕色)、白色文字、圆角、内边距。
 */
private void styleButton(Button btn) {
    btn.setFont(Font.font("Microsoft YaHei", FontWeight.BOLD, 14));
    btn.setStyle("-fx-background-color: #8F7A66; -fx-text-fill: white; "
        + "-fx-padding: 10 20 10 20; -fx-background-radius: 5;");
}

// ===== 以下为供 Controller 调用的公有接口 =====

/** 获取场景图 — Controller 注册键盘事件需要 Scene 引用 */
public Scene getScene() {
    return scene;
}

/**
 * 更新棋盘显示 — Controller 每次移动后调用
 * 保存 grid 引用并触发重绘
 */
public void updateBoard(Tile[][] grid) {
    this.grid = grid;
    drawBoard();
}

/**
 * 更新分数显示 — 将当前分数和最高分反映到界面
```

```
* @param score 当前分数
* @param best 本局最高分
*/
public void updateScore(int score, int best) {
    scoreLabel.setText(String.valueOf(score));
    bestLabel.setText(String.valueOf(best));
}

/** 更新底部状态栏文字 */
public void showStatus(String text) {
    statusLabel.setText(text);
}

/**
 * 显示获胜对话框
 *
 * 使用 JavaFX 的 Alert 对话框。
 * AlertType.CONFIRMATION 提供两个按钮: "继续"和"停止"。
 *
 * @return true=用户选择继续挑战, false=用户选择停止
 */
public boolean showWinDialog() {
    Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
    alert.setTitle("You Win!");
    alert.setHeaderText("恭喜你! 你达到了 2048! ");
    alert.setContentText("是否继续挑战更高分数? ");

    // 自定义按钮文字
    ButtonType yesBtn = new ButtonType("继续");
    ButtonType noBtn = new ButtonType("停止");
    alert.getButtonTypes().setAll(yesBtn, noBtn);

    // showAndWait() 阻塞等待用户点击, 返回 Optional<ButtonType>
    Optional<ButtonType> result = alert.showAndWait();
    return result.isPresent() && result.get() == yesBtn;
}

/**
 * 显示游戏结束对话框
 *
 * AlertType.INFORMATION 只有一个"确定"按钮
 */
public void showGameOverDialog(int score) {
    Alert alert = new Alert(Alert.AlertType.INFORMATION);
    alert.setTitle("Game Over");
    alert.setHeaderText("游戏结束! ");
```

```
        alert.setContentText("最终得分: " + score);
        alert.showAndWait();
    }

    // ===== 以下为棋盘绘制的私有方法 =====

    /**
     * 绘制整个 4x4 棋盘
     *
     * 绘制顺序:
     * 1. 先画深棕色底板（圆角矩形）
     * 2. 遍历 16 个格子，逐个调用 drawTile 绘制方块
     */
    private void drawBoard() {
        // 绘制棋盘底板 — 440x440 像素的深棕色圆角矩形
        gc.setFill(BOARD_BG_COLOR);
        gc.fillRoundRect(0, 0, BOARD_WIDTH, BOARD_WIDTH, 10, 10);

        if (grid == null) return;

        // 逐个绘制 16 个方格
        for (int i = 0; i < Board.SIZE; i++) {
            for (int j = 0; j < Board.SIZE; j++) {
                drawTile(i, j, grid[i][j].getValue());
            }
        }
    }

    /**
     * 绘制单个方块
     *
     * 绘制步骤:
     * 1. 根据行列号计算方块左上角坐标(x, y)
     * 2. 根据方块数值查配色表，绘制圆角矩形背景
     * 3. 如果是非空格子，绘制居中的数字文字
     *
     * @param row    行号 (0~3)
     * @param col    列号 (0~3)
     * @param value  方块数值 (0=空格不画数字)
     */
    private void drawTile(int row, int col, int value) {
        // 计算方块的位置 (间距 + 格号 × 格宽)
        double x = PADDING + col * (CELL_SIZE + GAP);
        double y = PADDING + row * (CELL_SIZE + GAP);
```

```
// === 步骤 1: 绘制方块背景 ===
gc.setFill(getTileColor(value));          // 查配色表获取颜色
gc.fillRoundRect(x, y, CELL_SIZE, CELL_SIZE, 8, 8); // 圆角半径 8px

// === 步骤 2: 绘制方块数字（仅非空格）===
if (value > 0) {
    gc.setFill(getFontColor(value));      // 文字颜色

    // 根据数字位数自适应调整字号
    int fontSize;
    if (value < 100)      fontSize = 40;    // 1~2 位数字（最大 99）
    else if (value < 1000) fontSize = 34;    // 3 位数字（128, 256, 512）
    else if (value < 10000) fontSize = 26;  // 4 位数字（1024, 2048）
    else                  fontSize = 20;    // 5 位及以上（4096, 8192...）

    gc.setFont(Font.font("Microsoft YaHei", FontWeight.BOLD, fontSize));

    // 计算文字宽度和高度用于居中定位
    // JavaFX 中需要用 Text 对象的 layoutBounds 来测量文字尺寸
    String text = String.valueOf(value);
    Text t = new Text(text);
    t.setFont(gc.getFont());
    double textWidth = t.getLayoutBounds().getWidth();
    double textHeight = t.getLayoutBounds().getHeight();

    // 计算居中位置：（格子尺寸 - 文字尺寸）/ 2 + 格子起始位置
    double textX = x + (CELL_SIZE - textWidth) / 2;
    double textY = y + (CELL_SIZE + textHeight) / 2 - 5; // -5 微调

    gc.fillText(text, textX, textY);
}
}

// ===== 颜色查询方法（静态方法） =====

/**
 * 根据方块数值获取对应的背景颜色
 * 如果数值超出配色表范围（如 4096），使用 2048 的金色作为默认色
 */
public static Color getTileColor(int value) {
    Color c = TILE_COLORS.get(value);
    return c != null ? c : TILE_COLORS.get(2048);
}

/**
```

```
    * 根据方块数值获取对应的字体颜色
    * 浅色背景(2,4)用深色字，深色背景(8 以上)用白色字
    */
    public static Color getFontColor(int value) {
        return value <= 4 ? FONT_COLOR_DARK : FONT_COLOR_LIGHT;
    }
}
```

1.7 GameController.java

```
package game2048;

import javafx.scene.input.KeyEvent;
import javafx.stage.Stage;

/**
 * GameController 类 — 2048 游戏的控制层（MVC 架构中的 Controller）
 *
 * 【类的职责】
 * 作为整个程序的中枢调度者，负责以下核心任务：
 * 1. 接收并解析用户输入（键盘按键 + 按钮点击）
 * 2. 调用 Model 层方法执行业务逻辑（移动、合并、生成、判定）
 * 3. 将执行结果同步到 View 层（刷新棋盘、更新分数、显示状态）
 * 4. 管理游戏流程（移动→更新→判定→提示的完整循环）
 * 5. 通过 DatabaseManager 将游戏数据持久化到 MySQL 数据库
 *
 * 【MVC 架构中的角色】
 * Controller 是 Model 和 View 之间的"中间人":
 * - Model (Board + GameState + DatabaseManager) 不知道 View 的存在
 * - View (GameView) 不知道 Model 的存在
 * - 只有 Controller 同时持有两者的引用，负责协调
 *
 * 【新增：数据库功能】
 * 使用 MySQL 数据库实现：
 * - 最高分持久化（程序关闭后不丢失）
 * - 每局游戏记录保存（得分、最大方块、步数、是否获胜）
 * - 窗口关闭时自动保存最高分和本局记录
 *
 * @author 苏富杰
 * @version 2.0
 */
public class GameController {

    /** 棋盘模型 — 管理 4x4 网格和移动合并算法 */
```

```
private Board board;

/** 游戏状态模型 — 管理分数和胜负状态 */
private GameState state;

/** 游戏视图 — JavaFX 图形界面 */
private GameView view;

/** 数据库管理器 — 持久化最高分和游戏记录 */
private DatabaseManager db;

/** 上一步的分数值，用于撤销时精确恢复 */
private int prevScore;

/** 本局累计操作步数（有效移动的次数） */
private int steps;

/** 本局出现的最大方块值 */
private int maxTile;

/**
 * 构造方法 — 游戏初始化的入口
 *
 * 执行流程：
 * 1. 初始化数据库连接并加载历史最高分
 * 2. 创建 Model 对象（Board、GameState）
 * 3. 创建 View 对象（构建 JavaFX 界面）
 * 4. 注册事件处理器
 * 5. 配置窗口关闭事件（保存数据到数据库）
 * 6. 显示窗口并获取焦点
 *
 * @param primaryStage JavaFX 主舞台
 */
public GameController(Stage primaryStage) {
    // === 初始化数据库 ===
    db = new DatabaseManager();
    int savedBest = db.getHighScore(); // 从数据库加载历史最高分

    // === 初始化模型层 ===
    board = new Board();
    state = new GameState(savedBest); // 传入历史最高分
    steps = 0;
    maxTile = 0;

    // === 初始化视图层 ===
```

```
view = new GameView(this::newGame, this::undo);

// === 配置主舞台 ===
primaryStage.setTitle("2048 - Java 课程设计");
primaryStage.setScene(view.getScene());
primaryStage.setResizable(false);

// === 窗口关闭事件：保存数据到数据库 ===
// 关闭窗口时自动保存最高分和本局记录
primaryStage.setOnCloseRequest(e -> {
    if (steps > 0) {
        // 只有玩过至少一步才保存记录
        db.saveHighScore(state.getBestScore());
        db.saveGameRecord(state.getScore(), maxTile, steps, state.isWon());
    }
    db.close();
});

primaryStage.show();

// === 键盘焦点设置 ===
view.getScene().getRoot().setFocusTraversable(true);
view.getScene().getRoot().requestFocus();

// === 绑定事件 ===
view.getScene().setOnKeyPressed(this::handleKeyPress);
view.getScene().getRoot().setOnKeyPressed(this::handleKeyPress);

// === 初始刷新 ===
refreshView();
view.showStatus("新游戏开始！使用 ↑↓↔ 或 WASD 移动");
}

/**
 * 键盘按键事件处理器
 * 支持方向键（↑↓↔）和 WASD 键两套方案
 */
private void handleKeyPress(KeyEvent e) {
    switch (e.getCode()) {
        case LEFT: case A: processMove(Direction.LEFT); break;
        case RIGHT: case D: processMove(Direction.RIGHT); break;
        case UP: case W: processMove(Direction.UP); break;
        case DOWN: case S: processMove(Direction.DOWN); break;
        default: break;
    }
}
```

```
}

/**
 * 处理一次完整的移动操作
 * 这是 Controller 最核心的方法，串联了游戏主循环
 */
private void processMove(Direction dir) {
    if (state.isGameOver()) return;

    prevScore = state.getScore();
    board.saveSnapshot();

    boolean changed = board.move(dir);

    if (changed) {
        // 累加分数
        int gained = board.getScoreGained();
        state.addScore(gained);

        // 更新本局最大方块值
        updateMaxTile();

        // 步数加一
        steps++;

        // 生成新方块、刷新界面、检查胜负
        board.spawnTile();
        refreshView();
        checkGameState();
    }
}

/**
 * 更新本局最大方块值
 * 遍历棋盘找到当前最大的方块数值
 */
private void updateMaxTile() {
    for (int i = 0; i < Board.SIZE; i++) {
        for (int j = 0; j < Board.SIZE; j++) {
            int v = board.getTile(i, j).getValue();
            if (v > maxTile) {
                maxTile = v;
            }
        }
    }
}
```

```
}

/**
 * 检查并处理游戏胜负状态
 *
 * 获胜时：弹出对话框询问是否继续
 * 失败时：弹出对话框显示得分，保存记录到数据库
 */
private void checkGameState() {
    // === 获胜判定 ===
    if (board.hasWon() && !state.isWon() && !state.isKeepPlaying()) {
        state.setWon(true);
        refreshView();
        boolean keepGoing = view.showWinDialog();
        if (keepGoing) {
            state.setKeepPlaying(true);
            view.showStatus("继续挑战更高分数！");
        } else {
            // 选择停止：保存记录
            db.saveHighScore(state.getBestScore());
            db.saveGameRecord(state.getScore(), maxTile, steps, true);
            view.showStatus("恭喜获胜！点击「新游戏」重新开始");
        }
    }
}

// === 失败判定 ===
if (board.isGameOver()) {
    state.setGameOver(true);
    view.showStatus("游戏结束！点击「新游戏」重新开始");
    view.showGameOverDialog(state.getScore());
    // 保存记录到数据库
    db.saveHighScore(state.getBestScore());
    db.saveGameRecord(state.getScore(), maxTile, steps, state.isWon());
    refreshView();
}
}

/**
 * 开始新游戏
 * 重置棋盘、分数、步数和最大方块值
 */
private void newGame() {
    board.reset();
    state.reset();
    steps = 0;    // 重置步数
```

```

        maxTile = 0;    // 重置最大方块
        updateMaxTile(); // 重新计算初始棋盘的最大方块
        refreshView();
        view.showStatus("新游戏开始! 使用 ↑↓↔ 或 WASD 移动");
        view.getScene().getRoot().requestFocus();
    }

    /**
     * 撤销上一步操作
     * 恢复棋盘快照和分数, 步数不变(撤销不计入步数统计)
     */
    private void undo() {
        if (state.isGameOver()) return;
        board.undo();
        state.setScore(prevScore);
        refreshView();
        view.showStatus("已撤销上一步");
        view.getScene().getRoot().requestFocus();
    }

    /**
     * 同步视图与模型数据
     * 从 Model 读取数据并推送到 View 进行界面更新
     */
    private void refreshView() {
        Tile[][] displayGrid = new Tile[Board.SIZE][Board.SIZE];
        for (int i = 0; i < Board.SIZE; i++) {
            for (int j = 0; j < Board.SIZE; j++) {
                displayGrid[i][j] = board.getTile(i, j);
            }
        }
        view.updateBoard(displayGrid);
        view.updateScore(state.getScore(), state.getBestScore());
    }
}

```

1.8 DatabaseManager.java

```

package game2048;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

```

```
/**
 * DatabaseManager 类 — 游戏数据库管理（MySQL 版）
 *
 * 【类的职责】
 * 使用 MySQL 数据库实现游戏数据的持久化存储，包括：
 * 1. 最高分记录的存取
 * 2. 每局游戏结束时的完整记录保存（分数、最大方块、步数、是否获胜）
 * 3. 历史记录查询
 *
 * 【为什么选 MySQL？】
 * MySQL 是目前最流行的关系型数据库之一，广泛应用于各类应用系统。
 * 使用 MySQL 能够展示 JDBC 数据库编程能力，包括连接管理、
 * PreparedStatement 防 SQL 注入、事务处理等核心技能。
 *
 * 【数据库表结构】
 * - game2048.high_score: {id, score, update_date}
 * - game2048.game_record: {id, score, max_tile, steps, won, play_date}
 *
 * 【面向对象设计要点】
 * 数据库操作全部封装在此类中，Controller 只需调用简单的方法，
 * 不需要关心 SQL 语句细节和连接管理。体现了信息隐藏和单一职责原则。
 *
 * @author 苏富杰
 * @version 3.0
 */
public class DatabaseManager {

    /** MySQL 数据库连接 URL */
    private static final String DB_URL = "jdbc:mysql://localhost:3306/game2048";

    /** 数据库用户名 */
    private static final String DB_USER = "root";

    /** 数据库密码 */
    private static final String DB_PASSWORD = "Sufujie303011";

    /** 数据库连接对象 */
    private Connection connection;

    /**
     * 构造方法 — 建立 MySQL 数据库连接
     * MySQL 必须保证服务已启动，且 game2048 数据库已创建
     */
    public DatabaseManager() {
        try {
```

```
// 加载 MySQL JDBC 驱动
Class.forName("com.mysql.cj.jdbc.Driver");

// 建立连接
connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);

System.out.println("MySQL 数据库连接成功");

} catch (Exception e) {
    System.err.println("数据库连接失败: " + e.getMessage());
    connection = null;
}
}

/**
 * 获取历史最高分
 * @return 历史最高分, 无记录则返回 0
 */
public int getHighScore() {
    if (connection == null) return 0;

    String sql = "SELECT MAX(score) FROM high_score";
    try (Statement stmt = connection.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {
        if (rs.next()) {
            int score = rs.getInt(1);
            return rs.wasNull() ? 0 : score;
        }
    } catch (SQLException e) {
        System.err.println("查询最高分失败: " + e.getMessage());
    }
    return 0;
}

/**
 * 保存最高分
 * 只有当新分数高于已有最高分时才插入
 */
public void saveHighScore(int score) {
    if (connection == null) return;

    int current = getHighScore();
    if (score <= current) return;

    String sql = "INSERT INTO high_score (score) VALUES (?)";
    try (PreparedStatement pstmt = connection.prepareStatement(sql)) {
        pstmt.setInt(1, score);
        pstmt.executeUpdate();
    }
}
```

```
        System.out.println("新高分已保存: " + score);
    } catch (SQLException e) {
        System.err.println("保存最高分失败: " + e.getMessage());
    }
}

/**
 * 保存一局游戏的完整记录
 * @param score    最终得分
 * @param maxTile  本局最大方块值
 * @param steps    操作步数
 * @param won      是否获胜
 */
public void saveGameRecord(int score, int maxTile, int steps, boolean won) {
    if (connection == null) return;

    String sql = "INSERT INTO game_record (score, max_tile, steps, won) VALUES (?, ?, ?, ?)";
    try (PreparedStatement pstmt = connection.prepareStatement(sql)) {
        pstmt.setInt(1, score);
        pstmt.setInt(2, maxTile);
        pstmt.setInt(3, steps);
        pstmt.setInt(4, won ? 1 : 0);
        pstmt.executeUpdate();
        System.out.println("游戏记录已保存: 得分=" + score + ", 最大方块=" + maxTile + ", 步数=" + steps);
    } catch (SQLException e) {
        System.err.println("保存游戏记录失败: " + e.getMessage());
    }
}

/**
 * 查询最近 N 局游戏记录
 * @param limit 返回记录条数
 * @return 记录列表
 */
public List<String> getRecentRecords(int limit) {
    List<String> records = new ArrayList<>();
    if (connection == null) return records;

    String sql = "SELECT score, max_tile, steps, won, play_date FROM game_record ORDER BY id DESC LIMIT ?";
    try (PreparedStatement pstmt = connection.prepareStatement(sql)) {
        pstmt.setInt(1, limit);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                int score = rs.getInt("score");
                int maxTile = rs.getInt("max_tile");
```

```
        int steps = rs.getInt("steps");
        boolean won = rs.getInt("won") == 1;
        String date = rs.getString("play_date");
        String status = won ? "获胜" : "未完成";
        records.add(String.format("[%s] 得分:%d 最大方块:%d 步数:%d %s",
            date, score, maxTile, steps, status));
    }
}
} catch (SQLException e) {
    System.err.println("查询记录失败: " + e.getMessage());
}
return records;
}

/**
 * 关闭数据库连接
 */
public void close() {
    if (connection != null) {
        try {
            connection.close();
            System.out.println("数据库连接已关闭");
        } catch (SQLException e) {
            System.err.println("关闭连接失败: " + e.getMessage());
        }
    }
}
}
```

评 语：

指导教师

日期

附

《面向对象程序设计（JAVA）》综合实验及报告要求

一、综合实验目的

通过《面向对象程序设计（JAVA）》综合实验，进一步巩固 JAVA 面向对象程序设计知识，熟悉程序设计的基本思想，掌握程序设计方法和编程技术，为学习后续课程、培养程序设计实际能力打下良好基础。

二、综合实验内容和要求

根据《面向对象程序设计（JAVA）》综合实验目的和要求，学生可以自拟综合实验题目，或由教师指定题目，题目的难度和程序设计工作量适中，学生通过课内和课外学习、编程综合训练，按时独立完成《面向对象程序设计（JAVA）》综合实验任务，并按规范要求撰写《面向对象程序设计（JAVA）》综合实验报告。

选题参考：学生成绩管理系统、**学生通讯录管理系统**、图书馆管理系统、个人财务管理系统、飞机订票信息系统、小型超市管理系统、网络通信程序等。

如《XX 管理系统设计》内容包括：

1. 系统需求描述
2. 系统总体设计
 - 1) 问题分析；
 - 2) 设计思路；
 - 3) 程序结构
3. 系统详细设计
 - 1) 界面设计、主菜单设计方法；
 - 2) 创建录入信息方法
 - 3) 数据库设计、使用；
 - 4) 信息显示、查询方法
 - 5) 添加信息方法；
 - 6) 修改信息方法
 - 7) 删除信息方法；
 - 8) 统计计算及排序方法
 - 9) 主方法

三、系统设计与实现

1. 概要设计

功能模块，本程序中用到的所有方法的定义、主程序的流程以及各程序模块之间的层次(调用)关系。

2. 详细设计

实现概要设计中定义的所有模块，对每个操作需要写出算法；对主程序和其他模块也都需要写出算法；可采用流程图进行描述，画出方法的调用关系图。

四、调试分析

内容包括：

-
- a. 调试过程中遇到的问题是如何解决的，以及对设计与实现的回顾讨论和分析；
 - b. 经验和体会等。

五、用户使用说明

使用说明、文档，如何使用所开发的系统，详细列出每一步的操作步骤。

六、测试结果

列出系统的测试结果，包括输入和输出。这里的测试数据应该完整和严格，最好多于需求分析中所列。

七、总结

总结所开发系统的优缺点，在综合实验过程中遇到的问题和解决办法，体会。

八、参考文献

九、附录：

程序代码

综合实验报告要求 3000 字数以上。